# FoodBridge — React Frontend Mentorship

Day 4 Notes — AuthContext, JWT Decoding, and Login Persistence

## 0. Axios Revision (Warm-up)

Recapped axiosInstance (avoids repeating base URL, easy to change later) and the .then()/.catch() pattern (Promise resolves after network delay; .then handles success, .catch handles failure). Also clarified: Axios auto-converts a plain JS object to JSON and sets Content-Type: application/json automatically — unlike fetch, where this must be done manually.

## 1. Why Context API Is Needed Here

Login's JWT/user data only existed locally inside Login.jsx and disappeared once that function finished. Multiple unrelated parts of FoodBridge need this same data: Navbar (show correct links per role), ProtectedRoute (block access if role doesn't match), axiosInstance (attach JWT to protected requests), and every Dashboard (know which role's data to fetch). Context API shares this data without manually passing it through props at every level (avoids prop drilling).

## 2. Building AuthContext.jsx

```
import { createContext, useState } from "react"

export const AuthContext = createContext()

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null)

  return (
    <AuthContext.Provider value={{ user, setUser }}>
      {children}
    </AuthContext.Provider>
  )
}
```

### Key concepts clarified:

- createContext() returns an object (like a container/box). That object has a built-in .Provider component inside it.
- AuthContext (the object) cannot be rendered directly as JSX — only AuthContext.Provider (the component inside it) can be.
- AuthProvider is a component WE wrote that holds the actual user state (useState) and internally renders AuthContext.Provider with the real value — it does the real work.
- value={{ user, setUser }} — outer braces = JSX expression syntax; inner braces = a plain JS object (shorthand for { user: user, setUser: setUser }).
- {children} — renders whatever was nested inside ... when it's used; those nested components can then read from context.

## 3. Named Exports vs Default Export

```
// Default export — only ONE per file, import with any name, no braces
export default function Login() { ... }
import Login from "./Login"

// Named export — MANY per file, import with exact name, in braces
export const AuthContext = createContext()
export const AuthProvider = ...
import { AuthContext, AuthProvider } from "./AuthContext"
```

AuthContext.jsx needs two separate exports (AuthContext itself, and the AuthProvider wrapper component), so named exports were required. Also standardized on arrow-function syntax for all components going forward.

Mistake caught: export AuthProvider = (...) => {...} is invalid — export must be followed by a proper declaration (const/let/function/class), not a bare assignment.

## 4. Activating AuthProvider in main.jsx

```
import { AuthProvider } from './context/AuthContext'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <AuthProvider>
      <App />
    </AuthProvider>
  </StrictMode>,
)
```

Mistake caught: initially wrapped App with the raw AuthContext.Provider instead of the custom AuthProvider component. That would have created a provider with no value/state attached at all, bypassing all the logic written inside AuthProvider.

## 5. Reading Context with useContext (in Login.jsx)

```
import { useContext } from "react"
import { AuthContext } from "../../context/AuthContext"

const Login = () => {
  const { user, setUser } = useContext(AuthContext)
  ...
}
```

useContext(AuthContext) reaches into the context and pulls out whatever was passed via value={...} in AuthProvider — here, { user, setUser } — then destructures both into local variables.

Mistake caught: useContext was initially called OUTSIDE the Login component (top-level in the file). React hooks can only be called INSIDE a component function — calling them outside breaks React's rules of hooks and throws an 'Invalid hook call' error.

## 6. Decoding the JWT with jwt-decode

Backend's /users/login returns a raw JWT string, not a wrapped user object. Installed jwt-decode to safely extract the payload instead of manually parsing it.

```
npm install jwt-decode
import { jwtDecode } from "jwt-decode"

.then((response) => {
  const token = response.data
  const decoded = jwtDecode(token)
```

```
    localStorage.setItem("token", token)

    setUser({
      token: token,
      role: decoded.role,
      email: decoded.sub
    })
  })
```

### Confirmed actual decoded token shape from backend:

```
{
  "sub": "karan.mehta2026@gmail.com",  // email
  "role": "DONOR",
  "iat": 1786439344,   // issued-at timestamp
  "exp": 1786525744    // expiry timestamp (for future auto-logout)
}
```

## 7. Persisting Login with localStorage

Problem: user state lives only in memory — a page refresh resets useState(null), logging the user out even though the JWT is still valid. Fix: save the token string in localStorage on login (localStorage.setItem('token', token)), since it survives refreshes and browser restarts.

## 8. Restoring Session on App Load (useEffect)

```
useEffect(() => {
  const jwt = localStorage.getItem("token")
  if (jwt) {
    const decoded = jwtDecode(jwt)
    setUser({
      token: jwt,
      role: decoded.role,
      email: decoded.sub
    })
  }
}, [])
```

useEffect(fn, []) — empty dependency array means the function runs exactly once, right after the component first mounts. This is the correct place to check localStorage on app startup.

Mistake caught: first version omitted the [] dependency array entirely, which makes useEffect run after EVERY re-render — wasteful and not the intended one-time-check pattern.

## 9. Important React Concept: State Updates Are Not Instant

console.log(user) placed immediately after setUser(...) printed null, even though setUser was called with real data. This is expected: setUser schedules an update, it does not apply immediately. The component must re-render before the new value of user is visible.

Verified correctly by moving console.log(user) OUTSIDE useEffect, into the component body (runs on every render). Observed: null printed twice (once for the real first render, once again due to React StrictMode's intentional double-invoke in development), followed by the actual decoded user object — confirming the restore-from-localStorage logic works correctly.

## 10. Milestone Reached

Login now stores { token, role, email } in global AuthContext, persists the token to localStorage, and automatically restores the logged-in user on page refresh via useEffect. This is the foundation for role-based Navbar links, ProtectedRoute, and attaching JWT headers to future API calls.

## 11. Git Commit (End of Day 4)

```
git add .
git commit -m "feat: implement AuthContext for global auth state with JWT decode and persistence

- Created AuthContext.jsx with createContext + AuthProvider (arrow function)
- Wrapped App with AuthProvider in main.jsx to activate context app-wide
- Connected Login to context via useContext, storing token/role/email in user state
- Installed jwt-decode to extract role and email (sub) from JWT payload
- Persisted token to localStorage on successful login
- Added useEffect in AuthProvider to restore user session from localStorage on app load
- Verified persistence survives page refresh (confirmed via console + StrictMode double-render beha
```

## 12. Next Steps (Day 5)

- Update Navbar to show different links based on user (guest vs Donor/NGO/Volunteer/Admin)
- Add a logout function (clear localStorage + setUser(null))
- Build ProtectedRoute component for role-based route access
- Redirect to correct dashboard after successful login based on role
- Attach JWT automatically to protected requests via Axios interceptor
- Connect Register form to POST /users endpoint (same pattern as Login)